

How to get Database Design Horribly Wrong

Robert Sheldon

Failure #1: Failing to plan

Not so long ago, planning seemed an obvious precursor to development. Database teams would no more build databases without careful planning than structural architects would erect buildings without well-considered blueprints. The belief was that a meticulous planning phase was the only way to ensure you didn't end up with a database hacked together with arbitrary components that failed to take into account data integrity and system performance. If you didn't invest the necessary time and effort in the beginning to understand the data, the constraints and the processes, you'd pay off big time with a database that would need to be refactored, replaced or scrapped altogether.

Although such planning wasn't always realized, it emerged as an ideal that many database people still adhere to today. That's not to say they can predict every design need in advance, but they still believe it's worth the effort to understand the data and how it will be used going forward. The last thing anyone wants is a database built for transaction processing when its intent is a data warehouse.

Times have changed, along with notions about how to go about database design. Agile methodologies have taken hold, forcing database teams to rethink their approaches to design and development.

In Agile, the Class-diagrams of Domain Modelling, derived from the Use-Cases, have usually replaced Entity-Relationship modelling; but the need for planning has not diminished. We still need to understand the data and what it's supposed to do and what are the best and safest ways to manage, store, and protect it.

In that respect, it's not the Agile methodologies that present many of the challenges faced by database teams, but rather having to work with individuals who do not understand the full nature of managing, storing, and protecting data. They see database development as comparable to application development, something that can be easily integrated within existing tools, systems, processes, schedules, and workflows, most of which were designed for application development. Although there are practices that database teams can benefit from, such as source control and continuous integration, database design and development are very different disciplines and need to be afforded the respect they deserve.

Ideally, the entire product team will appreciate the important role data plays and how it underpins all their other efforts. We live in the age of data. It drives our businesses and economies and is what keeps us employed. Our applications provide the means to display and manipulate data, but it is the data itself that gives those applications meaning. Yet some treat the database as merely a component of the application, without fully grasping the data's central role, compartmentalizing it in the same way they do dynamic libraries or web services-and careful database design often becomes part of the collateral damage.

Way out of Failure #1

The need for properly planning a database, whether done mostly upfront through a more traditional approach or in stages through the Agile approach, does not go away. In some cases, database teams are put into the position of having to evangelize the importance of proper planning and convincing those around them that taking a piece-meal approach to database design can result in a garbage database and unreliable data, or a lot of data migration planning to address data concerns that were ignored for short-term gains.

A product team, whether or not they've adopted Agile methodologies, must be willing to invest the resources necessary to properly understand the data and have a plan for managing, storing, and protecting it. The plan will change, of course, but those changes must also be as carefully designed and managed. The importance of the data is too great for it to be short-changed. The goal is to have a comprehensive vision of how to move forward that makes sense in relation to the data itself, whether the scope of that vision expands throughout the entire product lifecycle or is limited to the next few sprints.

Teams committed to Agile methodologies acknowledge the importance of modelling the domain, but they might find the planning process to be most challenging in the early stages. Data architects can certainly adjust their approach to design-perhaps not thinking on such grand scales initially-but domain knowledge of the data and how it will be used is still essential, and time and resources must be allocated to ensure that the product team has the understanding it needs in order to provide a

proper starting point and move forward in a coherent fashion. If that can happen within the Agile structure, great. If it needs to happen before the first sprint kicks off, that's fine too. But it must happen.

Failure #2: Not bothering with documentation

If a database and the related data environment are not properly documented, it will be difficult to maintain or alter them as business requirements change.

I was tempted to make this the number one failure, not that the failures are necessarily listed in order of priority, but the task of properly documenting a database is so often glossed-over that even discussions around planning and design fail to mention it. I can understand that, though. In the strictest sense, documentation has nothing to do with the design itself, but it has everything to do with communicating that design and making it accessible in the future.

But let me take a step back. When I say “documentation,” I’m not just talking about a modeling diagram. Sure, that’s a good thing to have, but so are data dictionaries, extended properties, data flow diagrams, details about the data sources and how data is transformed, and information about any other component and process that make up a data-driven project. Even the database’s object names play an important role (which we’ll cover in the next failure).

Unfortunately, proper documentation is often treated as an afterthought, if addressed at all. When schedules and budgets grow tight, documentation gets tossed out the window. But make no mistake, this is technical debt at a very high rate of interest. Those corners being cut now will soon cost an organization over and over again as players try to track changes, identify problems, fix bugs, implement new systems, or in any way make sense of the existing logic and what’s happening with the data.

Lack of proper documentation might be a result of shortsighted thinking or simply failing to understand the significant role it plays, but its role cannot be underestimated. It can serve as a valuable resource at every stage of the product lifecycle for a wide range of individuals, whether database developers, application developers, testers, analysts, managers, DBAs, data architects, new hires, or any other participants who would otherwise spend hours trying to make sense of something that should have been documented in the first place.

Way out of Failure #2

Documentation can, and should, come in many forms. A data dictionary can be a good place to start. It provides descriptions of the tables, columns, relationships, constraints, and other database objects. The data dictionary should be complete and accurate and kept up-to-date as components change. Some teams might prefer to use extended properties, if their RDBMS supports them, or use both, just as long as developers, DBAs, architects and everyone else can know what’s going on and have the same understanding of the purpose of each object, whether it’s a foreign key or check constraint or even a table. Even details such as why a constraint can’t be implemented can be useful.

Modeling diagrams can also be extremely helpful for conveying the database’s design to various players. Such diagrams not only show the objects within a database, but also the relationships between them.

These days, data modeling tools make it relatively easy to generate data dictionaries and modeling diagrams and update them as needed, again providing an accurate picture of the database at any given point in time and helping to ensure that everyone has the same understanding of how things work. Perhaps the biggest challenge with this approach is getting the team to work together to provide a consistent way of generating the information. That, of course, will depend on the team.

In addition to the database itself, your documentation should also include diagrams and detailed specs where appropriate that explain how all the pieces fit together. Too often, members of the database team have different perspectives about which databases are doing what, where the data comes from, and how it’s getting there. The documentation provides a single vision of the entire operation. The documentation should include details about data sources, extract/load operations, transformation processes, data flows, and any other relevant information so participants have a comprehensive view into the database and its data, and how all the pieces fit together to make everything work. And it doesn’t have to be pages and pages of Word documents that few will ever read. Sometimes a few good Visio diagrams is all it takes.

Failure #3: Using lousy naming conventions

The topic of naming conventions might not seem all that important as a design consideration, but if you’ve ever had to weed through a database with table names `st_cu` or `storc` (for “store customer”) or column names such as `fst_nm` or `Fst` (for “first name”), you know the frustration of trying to make sense of the schema and queries. Object names provide the first inroads into understanding a database. They serve as an introduction to the data and should make logical sense.

Inconsistent naming conventions and poor name choices serve no purpose but to frustrate those who need to access the database. For example, if a database uses some form of “ID” as part of the name for the artificial key, usually the primary key columns (such as **CustomerID**, **EmployeeID**, and **ProductID**), but some tables follow different conventions (such as **region_number**, **vendor_no**, or **Shipping Number**), developers, analysts, and DBAs will likely end up wasting time trying to figure out why these exceptions exist and what the logic is behind them, especially if the object names are even less descriptive than those shown here. Such practices can also lead to more errors in the code because the naming is no longer intuitive.

What can be even more frustrating is to come across names that must be delimited because they include spaces or reserved keywords, such as **check** or **statistics** or **verified stats**. There’s no reason to do this. Having to delimit identifiers only makes coding more frustrating and more difficult to read.

Way out of Failure #3

Part of your design planning should be to come up with a consistent naming standard that’s used throughout the database. There have been plenty of discussions on database forums about what that standard should be, such as whether to use underscores or camel case or take another approach. I won’t get into that argument here. What’s important is that you follow a consistent naming standard throughout and you use descriptive names so anyone who views the schema and queries can easily pick up on what’s going on. You might follow a convention such as **customerID**, **EmployeeID**, and **ProductID**, or go with **customerId**, **employeeId**, and **productId**, or take the underscore approach: **customer_id**, **employee_id**, and **product_id**. Just be consistent (even if that means being consistently bad to comply with an existing standard within the organisation).

One issue that often comes up is whether to prefix object names with metadata abbreviations that describe the object, such as the **tblProduct** table or the **vwEmployee** view or the **spGetCustomers** stored procedure. This is another area that tends to generate a fair amount of discussion. Few database professionals are in favour of this approach, at least for tables or views. In this case, object types can change, such as dropping a table and replacing it with a view to support a legacy application. Even so, some people believe strongly in their use, or at least partial use for some object types. That’s fine. Again, what’s most important is to be consistent and descriptive.

One other point: Try not to go overboard with the length of your names. Sure, systems such as SQL Server let you create identifiers up to 128 characters, but here’s what only 73 characters look like: **SalesGeneratedByCustomersAroundTheWorldWhoBoughtOurProductsViaTheInternet**. How much easier to call the table **InternetSales**. This, of course, is a bit of an exaggeration (I hope), but you get the point. Use a little common sense. Even abbreviations are okay if they’re universal. After all, who doesn’t know what **ID** means?

Failure #4: Not knowing how to normalize

Perhaps you’ve run into this situation. A program manager arrives on the scene in the early stages of database design and announces that you should start with a denormalized structure now and save a lot of trouble down the road. Such individuals are apt to cite their own experiences when denormalizing paid off in big performance dividends, their conclusions short on details and long on smug satisfaction, leaving little room for a reasoned debate.

Normalization is often seen by non-database types as a way to over engineer, add complexity, and ensure job security in coming years. This is as absurd as saying that network security has no purpose other than to make everyone’s life miserable. Normalization serves as the foundation on which SQL and relational database management systems (RDBMS) are built. Normalization provides the structure necessary to effectively access, modify, and manage the data, while ensuring its integrity over the long term.

A properly normalized database can also help improve performance, contrary to what denormalization advocates might suggest. Denormalizing too often or too soon can result in overly complex queries and locking contention as well as put the data’s integrity at risk. A normalized data structure is integral to effective database design and deviating from this strategy should be done only after carefully analyzing the situation and weighing the available alternatives.

Way out of Failure #4

Start by learning the rules of normalization and their importance. If you’re designing relational databases without knowing the difference between normal forms and Excel workbooks, stop what you’re doing and learn. If you need a kick-start, a great resource is the book “[Pro SQL Server 2012 Relational Database Design and Implementation](#),” by Louis Davidson and Jessica Moss.

You need to understand how normalizing works before you can design databases effectively. It’s not uncommon to find databases in production that wouldn’t meet the requirements of the First Normal Form, treating fields like arrays rather than atomic units, like that in the following table.

FranchiseID StoreLocations

101	Chicago, London, New York, Paris, San Francisco
102	Seattle
103	London, Munich, Paris, Athens
104	New York, London

Clearly there is a problem. Imagine if the table contained many thousands of rows and you wanted to find the franchises with stores in London. And imagine trying to ensure the integrity of the data when it's stored in this way. The **StoreLocations** column is essentially an unlimited text field. It has to be to accommodate the variety of possible location combinations. It's very difficult to ensure the integrity of such an arrangement.

Unfortunately, some might turn to a solution such as the following to solve the atomic dilemma:

FranchiseID	City1	City2	City3	City4	City5
101	Chicago	London	New York	Paris	San Francisco
102	Seattle	NULL	NULL	NULL	NULL
103	London	Munich	Paris	Athens	NULL
104	New York	London	NULL	NULL	NULL

What do all these **NULLs** mean, anyway? Do they have stores only in the cities listed? Do you not know if there are stores in other cities? And think about what it would mean to retrieve a list of the franchise's cities. For each franchise, you would have to query each column to determine whether a city exists. For a few hundred rows, that's no big deal. For millions of rows, it's another story.

So learn what normalization means and apply those principles to your design. When normalizing your database, start by separating the data into logical groupings. Every table model should do one thing, each row represent one specific thing, and each column represents one and only one value. A value might have multiple components, such as a text string, geospatial value, or something like **point(x,y)**, but they are still treated like single values. At the same time, ensure that data integrity can be properly enforced through the system's built-in mechanisms, such as data types, foreign keys, check constraints, and unique indexes.

Failure #5: Going overboard with normalization

Debates around normalization often center on query performance, forgetting that normalization serves additional purposes, particularly with regard to data integrity. In addition, these discussions often fail to consider other performance-related issues, such as indexing, contention, bandwidth, and system resources. It's easy to play the blame game with normalization because it's an abstract enough concept to warrant contempt, but in many cases, it's the wrong target.

Even so, the debate brings up an important point: Database design should take into account a number of considerations, including the queries accessing the data. If you have to join 20 tables to retrieve a single customer name, you might have taken normalization a step too far. Yet even that might be okay under certain circumstances.

Way out of Failure #5

What is "too far" is often debatable, and getting the normalization levels right can take practice and a bit of trial-and-error. As a general rule, you should aim to achieve at least the Third Normal Form. If it makes sense to take it to the Fourth and Fifth levels, do so. You're better off starting with a highly normalized structure and having to back off, than vice versa. At the same time, try to bring a little balance to the process.

Suppose you're working with a set of phone numbers. In theory, each component of a number represents an atomic unit: the country code, area code, exchange, and subscriber number. Should you separate each component into its own column, as in the following table?

PhoneID	CountryCode	AreaCode	Exchange	SubscriberNumber
101	1	206	555	2787
102	1	312	555	3456
103	1	415	555	7368

PhoneID CountryCode AreaCode Exchange SubscriberNumber

104 1 501 555 9923

105 1 612 555 8247

What about countries with different number structures? Or should you build a separate table for each component type to avoid redundancy? What if you have only a thousand numbers? What about several billion? How will the data be accessed? Updated? Does the interface simply require the entire phone number or are you supporting complex sorting and analytical operations?

Remember to keep your customer in mind when normalizing your database. If the requirements state that the customer wants *only* to be able to access a contact's email address, phone number, and name-each in its entirety-then that is what you should give them. They are not asking for complex scenarios that require you to split out the parts of the phone number or separate the email username from the domain or break out the first, middle, and last names. They want nothing but the three complete values.

Such as design might not keep the normalization gods happy or bold well for data integrity, but you're giving what the customer is asking for, which is a far cry from over normalizing the structure. You might be able to convince your clients to cede a little, such as separating the first names out from the lasts, but they're the ones to decide. If they come to you after the fact and ask for complex analytics, that is the time to point to the requirements that everyone agreed upon.

Normalization is a great tool, and is still an admirable goal to strive for, but so is common sense. Part of database design is to understand the data and how it will be accessed and used. You must take into account the application, its queries, the environment, and the users being served. In some cases, too much normalization is just that: Too much.

Failure #6: Assigning incorrect data types

Selecting the correct data type seems a foregone conclusion for anyone designing a database, but quite often a database is built with poor type choices, losing one of the most basic database mechanisms for ensuring integrity. Configuring your columns with the right types can also lead to improved query performance by avoiding implicit conversions, such as converting character data to integers

One common scenario is to go with types that are too large, just in case you might need them in some unforeseeable future, such as using **BIGINT** when **INT** will do. Although the 4-byte difference doesn't seem like much when considered on the per-value level, 4 bytes multiplied by 1 billion rows translates to nearly 4 GB of unnecessary storage and scanning.

Even differences such as using **CHAR(4)** rather than **CHAR(2)** can add up. The use of **NCHAR** and **NVARCHAR** can also point to wasted resources if **CHAR** and **VARCHAR** provide what you need. Why double the load unnecessarily? It's not just wasted disk space. All that extra size strains memory resources during queries and generally makes the query engine work harder.

But data type problems are not always limited to going too big. Choosing types that are too small can lead to errors and unexpected truncations. You might, for example, configure a column with **NVARCHAR(20)** based on the current set of data. However, three months later, values with greater lengths might start showing up. In some cases, you might have had no way to know they were coming, but chances are, if you had done your homework and studied the source systems properly and better understood the possible types of data, you would have been prepared for this eventuality.

Also be aware that the inclination to use an inappropriate data type might be pointing to a larger design issue. Consider a common lookup table that tries to handle lots of unrelated attribute pairs (discussed more in the next failure). If you find yourself trying to make one type fit all, you likely have a bigger problem.

Way out of Failure #6

The way out, of course, is to assign the most appropriate type to each column in your database. This, however, goes back to the need to fully understand your data and where that data going. It is part of the planning process and requires careful analysis. For example, if you're working with SQL Server and you know that the integers you'll be adding to a column will never exceed 32,000, then use **SMALLINT** rather than **INT**. Match the domain of the data you want to store to the data type that best fits (and then add a constraint to get down to exactly the domain you want).

Not only must you understand the data, but you must also understand the nature of the data types, as they're implemented in your RDBMS. How do numeric types

differ from float types? When is it appropriate to use a character type rather than a date/time type? How will the spatial types port to other systems? What is the best way to store BLOB data? How is **BIT** data actually stored? If you find yourself always defaulting to **VARCHAR** and **INT** for most of your columns, you should probably take a closer look at your design, your data, and the data types available to your system.

Failure #7: Using common lookup tables

An all-too-common scenario is to create a common lookup table for dumping in attribute pairs, regardless of data types or how that data will be accessed. These catchall tables, despite their apparent simplicity, can make queries more complex and make it more difficult to enforce data integrity. For example, you might have a table that looks something like the following:

Category	Code	Description
1	1a	4000000
1	2b	478234
1	3c	89778800
2	1490	Road bike
2	1491	Mountain bike
3	abc	sold
3	def	on hold
3	ghi	shipped
4	NM	New Mexico
4	CO	Colorado
4	WA	Washington
4	OR	Oregon

The **Category** column represents your domain of data, which can take an integer data type. For both the **Code** column and **Description** column, you would likely use the **VARCHAR** or **NVARCHAR** data types to accommodate the various possible values. Even if the value of one of the categories is nothing but integers, they are stored as character data, possibly resulting in implicit conversions. Such a table makes it nearly impossible to implement other types of constraints. For example, imagine trying to define a check constraint on the **Description** column.

Sometimes this sort of table comes about because a database contains several tables with a similar structure (code and description), and putting them together in this way seems the most expeditious solution. The categories, in this case, would represent the original tables. Not only do these type of tables make it difficult to enforce data integrity, they can also grow to be quite large and be filled with lots of repetitive data. And what about the concurrency issues you might run into when numerous queries are trying to access the same table?

Way out of Failure #7

Don't do it. Remember that you're dealing with a relational database system, not a CSV text file. You should model your tables and columns based on the domain of the data. As mentioned earlier, every table model should do one thing, each row represent one specific thing, and each column represent one and only one value.

Failure #8: Getting the primary keys wrong

Primary keys often generate a fair amount of controversy, usually around the use of surrogate and artificial keys. That said, a table should have a primary key that uniquely identifies each row in the table. Where things get tricky is in trying to decide which columns to use and what types of values to include.

According to the rules of normalization, each table should contain a natural key that uniquely identifies each row in that table, which is another way of saying that every row should be different. Often it takes a number of columns to get there, but uniqueness must be guaranteed. However, the natural key and primary key don't have to be one in the same. In fact, often they're not, just as long as the table has a natural key.

Some folks, on the other hand, are fine with only an artificial key to provide uniqueness, considering this approach normalized enough. Unfortunately, the lack of a natural key can result in rows that might or might not be duplicates, with no way of telling whether they are.

For example, suppose you have a table that contains product data. The table includes a surrogate key defined with an integer data type, a column that contains product codes (a mix of alphanumeric characters), and several other descriptive columns. If relying only on the surrogate key to ensure uniqueness, it's possible for two rows to contain the same product code and descriptive information, even though they're considered unique rows because of the surrogate key. Are the rows referencing the same product, or has the information for one product been entered incorrectly? Some database teams are fine with this scenario, preferring a few duplicates over force-feeding a natural key into the table. For example, in cases of high-performance inserts, they might accept some duplicates and clean them up later. In-Memory OLTP, for example, allows only one unique constraint, making the approach necessary at times. Even so, there are those who would claim that a natural key is still necessary in order to ensure data integrity. In this example, the product code seems a likely natural key, but it's not always that clear-cut.

Although this is a debate we won't solve here, it does point an important issue: the potential for using the natural key as the primary key. The potential problem with this is that, if the natural key changes, which it's much more likely to do than the surrogate key, you could end up with a chain reaction that might result in changes having to be made all over the place. If the natural key is particularly large and made up of multiple columns with varying types of data (such as one that includes first and last names, date of birth, and email address), your queries can be impacted, especially when joining tables.

Way out of Failure #8

First off, be sure to define a primary key on each table unless you've determined beyond any doubt that it is detrimental to its use and purpose. It is hard to imagine what this would be, since even a logging table has a sequence of entries which could be considered a natural key. The main thing is to strive for unique rows, which are what make a relational database click.

In terms of the primary key itself, many database professionals advocate for the use of a single integer column. In most cases, this strategy will serve you well. The query often uses the index associated with the primary key to compare and look up data. Queries almost always join tables based on primary key columns, usually in conjunction with foreign keys.

Simpler primary keys generally equate with better performing queries. If the query engine has to weed through multiple columns or complex data not easily sorted, such as GUIDs, the engine has to work harder and queries take longer. An integer requires only four bytes, is fixed length, can be added incrementally, and includes no special characters or spaces. Integers can also be easily sorted and searched.

In some cases, you might want to use GUIDs, which offer the advantage of universality, but [cause index fragmentation](#), will bog down queries because they're not as easy to sort and search as integers, and take far more storage. You might also find that a multi-column primary key serves your purposes better than the single integer. That's fine. Just make sure you understand the potential performance trade-offs.

When in doubt, you're probably best pursuing the single-column integer approach. If it requires the use of a surrogate key, simply add a column to the table and populate it with unique integers. Although a surrogate key adds overhead, the performance gains are usually worth it. As with other issues, however, be sure to take into account the data itself, along with the requirements for the data and how it will be accessed and used.

Failure #9: Failing to implement proper data integrity

One of the most important functions of a RDBMS is to ensure the integrity of the data, but the system can do its job only if you've specifically implemented the mechanisms within the database designed to protect data. Not only do these include proper primary keys and data types, but also components such as foreign keys, unique constraints, and check constraints. We've already discussed the importance of data types and primary keys and how they can be misapplied within a database. The same is true for other types of protections, which are often misused or not used at all.

A good example of improper use can sometimes be found in the implementation of check constraints over foreign keys. Both serve their purposes and can be important to ensuring the data's integrity. In some cases, however, one might be better than the other. For example, if you want to limit acceptable values to a few choices or a well-defined range of values, a check constraint will likely serve your needs. For example, if you know your values must be red, orange, or yellow, or must fall between the range of 300 to 400, then a check constraint will likely serve you well.

However, if the acceptable values are based on a large data set or those values change over time, you're probably better off adding a table and defining a foreign key relationship. For example, if the values are product names that number in the thousands and change frequently, a domain table makes the most sense. The data is

easier to maintain, doesn't require a DDL change, and multiple tables can access it.

Although domain tables can add overhead, they can be a more effective means of enforcing referential integrity in such situations. In addition, they make it possible to reference the data from multiple places, rather than having to define redundant check constraints.

Of course, this is only one sort of problem. Often the bigger issue is not using any sort of constraint to limit the data, such as failing to use unique constraints when the data should be unique or not limiting values to only those that should be permitted, such as ensuring a column containing salary data is always greater than 0 so it won't break the client application. Even the use of default constraints can help ensure the data is as it should be.

Part of the problem is that some teams defer to the application to enforce integrity. The challenge with this approach is that it fails to take into account that a database can be accessed in multiple ways, often by numerous applications. Even if the app can get it right, multi-table checks such as foreign keys aren't possible due to concurrency. You would have to lock a lot of data to make sure no one violates your rules until you commit the transaction.

In addition, applications change and so do processes. Data should be protected where it resides: within the database. This doesn't mean the applications should forego their own protections, but you shouldn't rely on them to do the job that databases were designed to do, part of which is to ensure the integrity of the data.

Way out of Failure #9

The way out is, of course, to use the database's capabilities to ensure the data's integrity. Not surprisingly, this goes back to understanding the data and the types of values that can be expected and which of those should be permitted.

If an order should not be entered into a sales table unless the product in that order already exists, then you should use a foreign key relationship to enforce that business rule. If each product ID listed in the product table should be unique, even though it is not the primary key, then use a unique constraint to ensure that uniqueness. If product sizes should be limited to S, M, L, and XL, then use a check constraint or domain table to define those values. If an order's date should default to the current date, then use a default constraint to define that date.

Without these types of safeguards, you could end up with inaccurate and inconsistent data throughout your database. The mechanisms used to ensure that integrity of the data are as important as the tables that hold the data. Don't relegate integrity to the back burner. It should be part of the design process. And don't turn it over to the application. It is the job of the database to ensure that integrity and it is your job to make sure the logic is in place so the database can do what it was meant to do.

Failure #10: Not properly indexing

One could argue that indexing is not a database design issue, but rather a query optimization one, and query optimization should come only after you've delivered a solid data model. For the most part, I agree with this. It's too difficult to know in advance exactly what the customer will do. But some indexing decisions should be taken into account as part of the initial planning phase, with the understanding that things might have to change.

A poorly indexed database can cause all sorts of trouble, whether there are too few indexes or too many. It's better to think of proper indexing as an ongoing process, than a set of fixed structures, and there's nothing wrong with giving that process a kick-start when you initially roll out your database. You'll likely want to have at least some indexes in place; otherwise, you could end up with a bunch of heaps that are nearly impossible to access. Much of the work occurs around the primary keys and unique constraints, which are enforced under the covers as indexes.

Way out of Failure #10

Indexing is a complex topic and one that deserves much more coverage than what we can give it here. Still, you should still take into account a few basic indexing considerations when pushing forward with the initial design. The most notable of these concerns your primary keys and the indexes associated with them. Let's look at how it works in SQL Server to give you a sense of why this is a design consideration.

As we discussed earlier, you'll likely want to define a primary key on most, if not all, your tables. In SQL Server, when you create a primary key on a table, the database engine also creates a clustered index, unless one already exists elsewhere or you specifically ask it not to. A clustered index is essentially the table, with the data ordered based on the values in the indexed column (or columns). The clustered index does not contain a duplicate of the data. It is the data.

You can think of a table's clustered index as the table with the data sorted. The sorting is based on the indexed column (or columns). Because of this, the indexed

column becomes an important consideration when taking into account the queries that will access the data. In most cases, the queries will use that column to join the table to other tables or to access to the table data. For more information about SQL Server primary keys and clustered indexes, see Phil Factor's article "[Primary Key Primer for SQL Server](#)."

Because of the way clustered indexes work, you can define only one such index on a table. However, the clustered index does not necessarily need to be the primary key. As stated earlier, when designing your database, you should have a sense of how the data will be queried. You might determine that a different column would be better suited for the clustered index, in which case, you would define your primary key with a unique, nonclustered index. But you have to be careful which columns you choose for the clustered index. There are many considerations to take into account-too many to go into here-but you can see why you need to give some thought to this issue when planning your database.

As was also mentioned earlier, if you know the values in a column or set of columns must be unique and they're not the primary key, you should define a unique constraint. When you do this in SQL Server, the database engine also creates a unique index. In fact, there is no significant difference in SQL Server between creating a unique constraint and creating a unique index. The point is, if you're designing a database and you want to ensure uniqueness, indexing is again part of the equation.

Some database designers will also define indexes on their foreign key columns as part of the initial database definition. In most cases, you'll want to index these columns, but keep the larger picture in mind. You want to be careful not to over-index your database and you want to keep in mind that proper indexing should be part of the query optimization process. Indexes can be added and dropped, of course, but that comes with overhead, so give some thought to which columns you decide to index. Think about the cardinality of each relationship first and determine whether primary keys and alternative keys are already in place. You might want to have a discussion with your DBA before you decide to index all your foreign key columns.

Failure #11: Performing inadequate testing

Testing your design is as important as the design itself. The database represents the core of your entire system, no matter what type of applications it's supporting. Without proper testing, you risk having the entire system fail. This should be a no-brainer, right? If you're implementing a system as critical as a database, then QA should be banging away at it. Unfortunately, time and resource constraints can sometime lead to this step being skipped or severely curtailed. The result can be a disaster.

Way out of Failure #11

There's no getting around it. The database must be fully tested, and not just by running a few queries or offering up a few hundred rows of data. The tests should approximate the production environment as closely as possible, and that means hitting the database with everything you've got. When you're planning your database design, you must also be planning when and how it will be tested. The QA team will drive much of this, but you, or someone, needs to ensure that nothing is being missed. If the QA team and database team are not talking to each other, then something is wrong with the process.